

如果根据表达能力,形式化方法大致还可分为:模型方法、代数方法、进程代数方法、逻辑方法和网络模型方法等。

17.5.3 基于系统的构件测试方法

软件架构发展是基于软件构件思想的创立而快速发展起来的, 构件软件测试是保障和提高构件软件可靠性的重要手段。构件化设计思想已被嵌入式系统广泛应用, 由于嵌入式软件的开发成本是一般软件成本的三倍多, 因此, 解决嵌入式软件的重用、遗产软件继承等问题是摆在嵌入式系统开发人员面前的首要问题。基于系统的构件测试方法给软件重用的验证和确认提供了一种有效测试技术。

基于系统构件的测试方法的测试对象是软件构件，与传统测试相比，测试对象上发生了较大变化，因此测试目的就有所不同。与传统测试方法相比，其测试目的发生了变化。传统测试目的是暴露软件中隐藏的 errors 和缺陷、验证软件是否正确实现用户需求。而基于构件的测试目的如下：

- (1) 是否按照需求规约构造高质量的构件。
- (2) 是否按照构件标准和构件模型构造构件。
- (3) 是否选择并部署适合系统的可重用构件。
- (4) 是否在系统中正确地复用构件。
- (5) 是否正确修改或升级构件。

由于目的的差异,给构件测试带来了众多不确定问题,主要表现在:

- (1) 异构性。实现语言的不同、采用的操作系统不同和体系结构多种多样问题。
- (2) 源代码不可得性。通常, 构件提供者是以二进制形式交付的, 构件无源代码。
- (3) 演化性。构件在不断升级、增减, 其演化带来多版本的管理问题。
- (4) 不确定。一个输入序列可能会有不同的执行路径, 产生不同的结果。
- (5) 测试充分性准则难以确定。

针对构件测试的开发过程, 构件测试过程分为单元测试、集成测试、系统测试和回归测试。

图 17-23 给出了构件开发过程和构件测试过程的对应关系。

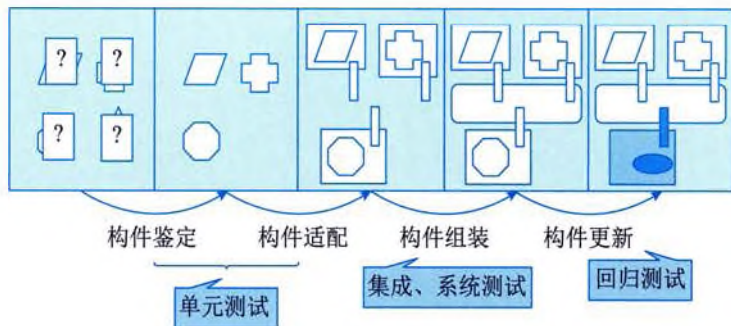


图 17-23 构件开发和构件测试过程的关系

- (1) 单元测试的主要目的是发现构件本身的错误，其常用技术包括随机测试、等价类测试、边界值测试和接口变异测试等。测试任务包括：

- 使用白盒测试确认构件的内部逻辑、数据和程序结构。
- 使用黑盒测试确认构件的外部功能、行为和接口。
- 度量其性能。
- 报告测试结果、程序错误和测试覆盖率。

(2) 集成测试是发现构件与应用软件交互的错误，其任务是：

- 构件之间的通信和交互关系。
- 需要关注的两个问题是构件的集成顺序和如何测试新集成的构件。
- 选择合适的构件，就需要考虑构件是否符合系统需求、构件质量是否足够好、构件对系统其他部分是否有影响。

(3) 系统测试是测试整个系统的功能与性能。

(4) 回归测试的主要任务是：

- 确定变化。构件规约、接口、设计、程序代码。
- 分析变化影响。构件功能、接口、结构、行为。
- 选择可复用的测试用例。
- 增加新的测试用例。

当然，基于系统的构件测试才刚刚起步，目前还没有比较实用的构件测试准则，仍然沿用传统软件测试的某些准则，如白盒测试、黑盒测试等，测试人员可根据实际情况提出自己的准则。